
M151 ENGINE — EXAMPLES/SIMPLE_ENGINE

The Simple Engine Guide

A guided tour of a small, complete reference game engine — what each system does, why it's shaped the way it is, and what to build next.

Companion to: `examples/simple_engine/`

Run it with: `python main.py`

Controls: WASD move · SPACE shoot · F1 hitboxes · ESC quit

Table of Contents

1	Why This Folder Exists	Prototype vs. engine
2	Folder Structure, Piece by Piece	What lives where, and why
3	The Camera & Coordinate Spaces	World, camera, screen
4	Input & Direction Normalization	Why diagonals $\times 0.707$
5	Entity, EntityManager, Scene, Game	The architecture that was missing
6	Animation & Collision	Two small, honest systems
7	Data-Driven Spawning	enemies.json and the factory pattern
8	Exercises	Six concrete ways to extend the demo
9	Where To Go Next	What a "full" engine still has that this doesn't

1 • Why This Folder Exists

Before this folder existed, M151_Engine had several working prototypes — `examples/entity_camera_relationship/main.py`, `games/main-with_enemy.py`, `games/01_Spawner/main-with_spawner.py` — each one teaching a real concept (camera math, sprite animation, JSON-driven spawning) but each one also *re-implementing its own game loop from scratch*. Every prototype had its own `pygame.init()`, its own hand-rolled list of enemies, its own copy-pasted collision check glued into the bottom of the update loop.

That repetition is the difference between a collection of prototypes and an **engine**. An engine is the part you don't rewrite every time you start a new game — a small set of systems that don't know or care what game you're building, plus a place for game-specific code to plug into them.

THE ONE RULE

`engine/` never imports from `entities/` or `scenes/`. Every system in `engine/` only ever touches generic `Entity` objects — `position`, `size`, `update()`, `draw()`, `get_hitbox()`. It has no idea a "Player" or "Enemy" exists. That's what lets `EntityManager` and the collision system work unchanged no matter what you build in `entities/` next.

This demo is deliberately small — one map, one player, four enemies, one weapon. The point isn't the game; it's that every system in it (camera, animation, collision, spawning, scenes) is written the way you'd want it if the game were ten times bigger, so you have a solid floor to build on for the next few weeks instead of a fourth throwaway prototype.

2 · Folder Structure, Piece by Piece

```
simple_engine/
├── main.py           entry point – ~5 lines, wires Game to PlayScene
├── engine/           reusable systems – copy into a new project as-is
│   ├── game.py       owns the window, the clock, and the run loop
│   ├── scene.py      base class for one game screen/mode
│   ├── entity.py     base class every game object inherits from
│   ├── entity_manager.py registry of all live entities, grouped by name
│   ├── camera.py     world ↔ screen coordinate conversion, edge-clamped scroll
│   ├── animation.py  picks the right sprite frame for the current direction
│   ├── collision.py   rectangle overlap checks between entity groups
│   ├── resource_loader.py loads + caches images, cuts sprite sheets
│   ├── input.py      polls WASD into a normalized 8-directional vector
│   ├── spawner.py    turns a JSON file into live entities via a factory fn
│   └── config.py     every tunable constant in one place
├── entities/        THIS game's objects, built on engine/entity.py
│   ├── player.py
│   ├── enemy.py
│   └── missile.py
├── scenes/
│   └── play_scene.py wires engine systems + entities into one playable scene
├── data/
│   └── enemies.json  enemy spawn positions / speeds / trails
├── assets/
│   ├── images/       art (see per-folder README.md for used vs. not-yet-wired)
│   ├── sounds/       empty – see assets/sounds/README.md
│   └── fonts/         empty – see assets/fonts/README.md
└── docs/
    └── M151_Simple_Engine_Guide.pdf  this document
```

engine/ — the reusable part

Every file here works with plain data (numbers, dicts, generic `Entity` objects) and pygame primitives. Nothing here imports `Player`, `Enemy`, or anything from `data/enemies.json`. If you started a brand-new game tomorrow, you could copy this whole folder over unchanged.

entities/ — this game's objects

`Player`, `Enemy`, and `Missile` are where game-specific behavior lives: what WASD does, what "patrol" means, how a missile picks its velocity. They all inherit from `engine.entity.Entity`, which is what lets them share `EntityManager` and the collision system.

scenes/ — wiring, not systems

`PlayScene` is deliberately the "messy" file — it's the one place allowed to know about both `engine/` and `entities/` at once. It loads images, builds the player and enemies,

and tells `EntityManager / Camera / CollisionSystem` what to do each frame. When you add a title screen or a pause screen, it'll be a new file in this folder, not a change to `PlayScene`.

data/ — tuning without touching code

`enemies.json` is the only file you need to edit to change which enemies exist, where they start, how fast they move, and which movement trail they use. See Chapter 7 for the exact format.

3 · The Camera & Coordinate Spaces

Three coordinate spaces show up everywhere in this engine, and keeping them straight is the single most useful mental model for any 2D game with a scrolling map:

- **World space** — where an entity truly lives, on the full map. The demo's map is **1920 × 5760 px**. The player spawns at its exact center: world **(960, 2880)**.
- **Camera space** — the rectangle of the world currently visible: a **960 × 540** window starting at (`camera.x` , `camera.y`).
- **Screen space** — where something actually gets drawn, in pixels from the top-left of the window.

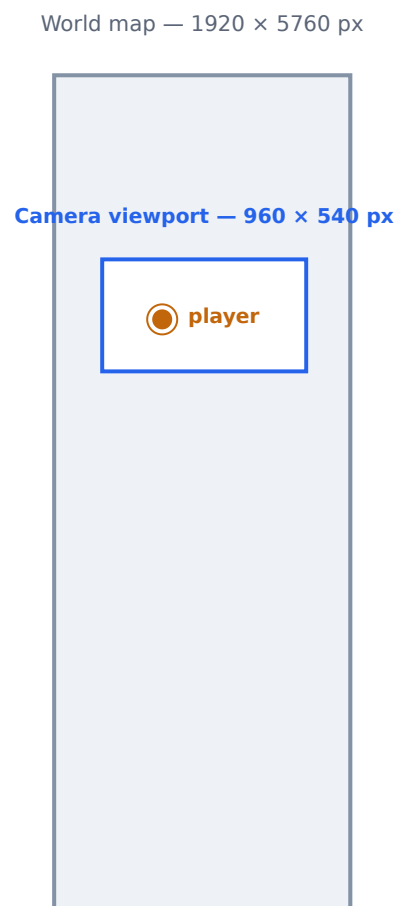


Fig. 3.1 — Only what's inside the camera rectangle is ever drawn to the screen.

The formula

```
def world_to_screen(self, world_x, world_y):  
    return world_x - self.x, world_y - self.y
```

WORKED EXAMPLE

Say the player has walked to world **(2100, 3080)**, and the camera's top-left corner is currently at world **(1632, 2810)**:

```
screen_x = 2100 - 1632 = 468
screen_y = 3080 - 2810 = 270
```

The sprite is blitted at pixel **(468, 270)** — just left of screen-center, exactly as you'd expect for a camera roughly centered on the player. This one line is the entire relationship between an entity and the camera: `Entity.draw()` never touches `camera.x / camera.y` directly, it just receives already-converted screen coordinates.

Why the camera has a dead-zone

`Camera.follow()` doesn't lock the camera exactly onto the player's center every frame — that makes tiny back-and-forth movements look jittery. Instead it only re-aims once the player drifts past the middle third of the screen horizontally (half/four-fifths vertically), then clamps the result so the camera never scrolls past the map edge:

```
self.x = max(0.0, min(target_x, float(map_w - self.screen_w)))
self.y = max(0.0, min(target_y, float(map_h - self.screen_h)))
```

Near a map edge, the camera "stops" at that clamp limit while the player keeps moving — the player visually slides toward the edge of the screen instead of staying centered. That's correct behavior, not a bug: it's the only way to avoid showing black space beyond the map.

WHY DRAW_MAP USES AREA=

```
surface.blit(map_surface, (0, 0),
              area=(int(self.x), int(self.y), self.screen_w, self.screen_h))
```

The `area=` parameter tells pygame "copy only this rectangle out of the source image." That keeps the per-frame cost fixed at 960×540 pixels no matter how large the map grows. The naive alternative, `surface.blit(map_surface, (-cam_x, -cam_y))`, draws the *entire* map every frame and lets pygame clip the off-screen part — it looks identical but does far more work for nothing.

4 · Input & Direction Normalization

`InputManager.read_movement()` polls WASD and returns a unit vector plus a direction name. The one non-obvious piece of math here is **why diagonal movement gets multiplied by 0.707**.

THE PROBLEM

Pure up movement (W only) gives $dx=0$, $dy=-1$, a vector of length $\sqrt{0^2 + 1^2} = 1.0$. But naive diagonal movement (W+D) gives $dx=1$, $dy=-1$ — a vector of length $\sqrt{1^2 + 1^2} \approx 1.414$. Multiplied by speed and dt unchanged, the player would move **41% faster** diagonally than in a straight line, purely as a side effect of the Pythagorean theorem.

THE FIX

Scale each diagonal component by $1/\sqrt{2} \approx 0.70710678$:

```
dx = 0.707, dy = -0.707
length =  $\sqrt{0.707^2 + 0.707^2} = \sqrt{0.5 + 0.5} = \sqrt{1.0} = 1.0$  ✓
```

Same magnitude as a cardinal move, so multiplying by `speed * dt` afterward produces equal real-world speed in all 8 directions.

```
match (up, down, left, right):
    case (True, False, True, False): return (-d, -d, config.UP_LEFT)
    case (True, False, False, True):  return ( d, -d, config.UP_RIGHT)
    ...
    case _:                          return (0.0, 0.0, None)
```

Matching on the 4-key tuple covers all 8 directions (4 cardinal + 4 diagonal) in one place. Opposing keys (W+S, or A+D) simply aren't in the match table, so they fall through to the default case: no movement, direction unchanged.

5 · Entity, EntityManager, Scene, Game

This is the architecture that no earlier M151_Engine prototype had, and it's the biggest structural upgrade in this folder.

Entity — one shape for everything

In the older prototypes, `Player` and `Enemy` inherited from `Entity`, but `Missile` did not — even though it had the exact same `world_x` / `world_y` / `update()` / `draw()` shape. That inconsistency is exactly why every prototype had to hand-write its own missile-vs-enemy check instead of a reusable "compare group A against group B." Here, **everything** that lives in the world — player, enemies, missiles, future pickups — inherits from `Entity`.

EntityManager — one registry, named groups

Before (per-prototype)	Now (EntityManager)
<pre>self.enemy — one hardcoded slot self.missiles — a bare list Each prototype rewrites its own update/draw/cleanup loop.</pre>	<pre>entities.add(obj, group="enemies") entities.update(dt, map_w, map_h) — one loop, any group entities.draw(surface, camera, draw_order=[...])</pre>

WHY KILL() IS DEFERRED

`Entity.kill()` only sets `alive = False`. Removal happens once per frame in `EntityManager.purge_dead()`, *after* collision checks. Mutating a list mid-iteration (`list.remove()` inside a `for` loop over that same list) silently skips the element right after the one you removed — a classic bug where a hit enemy "survives" every other frame. Deferred removal sidesteps the whole problem.

Scene — swappable game states

`Scene` is three empty hook methods: `handle_event`, `update`, `draw`. `PlayScene` is the only one that exists right now, but the mechanism for adding a title screen or pause menu is just: write a new `Scene` subclass, and set `game.scene` to an instance of it. No branching logic anywhere else has to change.

Game — the loop, written once

```
game = Game()          # pygame.init() + set_mode() happen here
game.run(PlayScene())  # PlayScene is only constructed AFTER that
```

WHY GAME() TAKES NO SCENE IN ITS CONSTRUCTOR

Loading an image with `Surface.convert()` requires a display mode to already be set. If you wrote `Game(PlayScene())` instead, Python would construct `PlayScene()` — which loads images — *before* `Game.__init__` gets a chance to run `pygame.init()`, and it would crash immediately. Building the window first, then handing the Scene to `run()` as a separate statement, guarantees the right order. This is a real ordering bug that came up while building this very demo — worth knowing before you hit it yourself.

6 · Animation & Collision

Animator

The art in this demo has exactly one frame per direction, so animation currently looks static. `Animator` is still built to support *lists* of frames per direction:

```
self._frames = {
    direction: (surfaces if isinstance(surfaces, list) else [surfaces])
    for direction, surfaces in frames.items()
}
```

TRY IT YOURSELF

Give `Enemy` a two-frame walk cycle by passing `{"right": [frame_a, frame_b], ...}` instead of a single `Surface` per direction — `Animator.update()` will start advancing frames automatically once it sees a list with more than one entry. Nothing in `Enemy`, `EntityManager`, or `PlayScene` needs to change.

Collision

`collision.check_groups(missiles, enemies, on_hit)` builds one list of hitboxes for group B, then checks every entity in group A against it with `pygame.Rect.collidelist()` — an $O(n)$ scan per query.

NOTE

$O(n)$ per query is completely fine at the entity counts this demo (and most learning projects) deal with — tens of entities, not thousands. A spatial grid or quadtree only starts to matter once you have hundreds of moving colliders on screen simultaneously. Building that before you can measure it being slow is solving a problem you don't have yet.

Collision hitboxes are always computed in **world space** (`Entity.get_hitbox()`), never screen space — so a hit registers correctly no matter where the camera happens to be looking. Press **F1** in the running demo to see these hitboxes drawn directly onto the screen via `collision.draw_debug_hitboxes()`.

7 · Data-Driven Spawning

`data/enemies.json` is the only file you should need to touch to change what enemies exist:

```
{
  "id": "alpha",
  "spawn_x_from_center": 260,
  "spawn_y_from_center": 0,
  "trail": "patrol",
  "speed": 180
}
```

Field	Meaning
<code>id</code>	Label printed to the console when this enemy is destroyed.
<code>spawn_x_from_center</code> <code>spawn_y_from_center</code>	Pixel offset from the map's exact center — not an absolute position.
<code>trail</code>	"patrol" (cycle all 8 compass directions on a timer), "bounce" (billiard-ball diagonal bounce off map edges), or "zigzag" (alternating horizontal/vertical legs).
<code>speed</code>	World pixels per second.

WHY OFFSETS FROM CENTER, NOT ABSOLUTE PIXELS

`JSONSpawner.load()` computes `world_x = map_w / 2 + spec["spawn_x_from_center"]`. Using an offset from the map's center instead of a hardcoded pixel position means the exact same `enemies.json` keeps working if you swap in a differently-sized map image — nothing has to be recalculated by hand.

The factory pattern

```
def load(self, key: str, factory) -> list:
    ...
    for spec in data.get(key, []):
        world_x = self._map_w / 2 + spec.get("spawn_x_from_center", 0)
        world_y = self._map_h / 2 + spec.get("spawn_y_from_center", 0)
        spawned.append(factory(spec, world_x, world_y))
    return spawned
```

`JSONSpawner` doesn't know how to build an `Enemy` — `PlayScene._make_enemy()` does, and gets passed in as `factory`. This is the general version of the ad hoc `EnemySpawner` in `examples/enemies_spawner/spawner-example.py`: because the "how do I build one"

logic is injected rather than hardcoded, the exact same `JSONSpawner` class could just as easily spawn pickups, obstacles, or anything else you invent, from a different JSON file.

8 • Exercises

Roughly in order of difficulty. Each one only touches one or two files.

- 1 Add a fifth enemy.** Add one more object to `data/enemies.json` with a new `id`, `offset`, `trail`, and `speed`. No code changes at all — this is the whole point of Chapter 7.
- 2 Add a fourth movement trail.** Add a `_update_spiral()` (or anything you like) method to `entities/enemy.py`, and one `elif` branch in `Enemy.update()`. Reference it from `enemies.json` as `"trail": "spiral"`.
- 3 Swap the map.** Point `engine/config.py`'s `MAP_IMAGE` at `assets/images/background/land/test-land.png` instead. Check that image's size and adjust the spawn offsets in `enemies.json` if enemies now land outside the map.
- 4 Give one enemy a full-image sprite** instead of a sheet-cut one. Load e.g. `assets/images/enemies/BossTank.png` directly with `ResourceLoader.load_image()`, and give `Enemy` an optional constructor argument to use a single static image instead of the direction-keyed `Animator`.
- 5 Add a title screen.** Write a `scenes/title_scene.py` with a `TitleScene(Scene)` that draws `assets/images/screens/Title ScreenText.png` and waits for a keypress, then update `main.py` to start on `TitleScene`. See Chapter 5 for why swapping scenes needs no changes to `Game`.
- 6 Add a sound effect.** Follow the steps in `assets/sounds/README.md` to build a minimal `engine/audio.py`, then call it from `PlayScene._fire_missile()` and `PlayScene._on_missile_hit_enemy()`.

9 · Where To Go Next

This engine is intentionally small. Here's what's missing compared to a "full" 2D engine, why it isn't here yet, and what adding it would actually involve — so you can pick the next thing deliberately instead of guessing.

Missing piece	Why it's not here yet	What adding it looks like
Sound	No sound assets exist in the project yet — building the system before you have a real file to test against means guessing at the API.	See <code>assets/sounds/README.md</code> and Exercise 6.
Title / pause / game-over screens	One <code>Scene</code> was enough to prove the architecture; a scene <i>stack</i> (for pausing without losing <code>PlayScene</code> 's state) is the natural next step once you have >1 scene.	See Exercise 5, then generalize <code>Game.scene</code> into a list you push/pop instead of a single reference.
Tile-based levels	The demo uses one big pre-rendered map image, which is simpler to reason about first. A tilemap (grid of small reusable tiles + a level format like Tiled's <code>.tmx</code>) is how most 2D engines actually handle large or procedurally varied worlds.	A new <code>engine/tilemap.py</code> that loads a tile atlas + a 2D grid of tile indices, and a <code>Camera.draw_map()</code> variant that draws visible tiles instead of one <code>area=</code> blit.
Spatial partitioning for collision	Not needed at this entity count — see the callout in Chapter 6.	A grid or quadtree that only checks entities in nearby cells, once you actually have hundreds of colliders and can measure the slowdown.
Save / load	Nothing yet is worth persisting between runs.	Serialize whatever state matters (player position, remaining enemies) to JSON — the same library <code>JSONSpawner</code> already uses.
Particle effects	Purely cosmetic polish; adding it before the core loop is solid would be decoration on an unfinished structure.	A <code>Particle(Entity)</code> with a fixed lifetime and no collision, managed by the same <code>EntityManager</code> group mechanism already in place.

M151 Simple Engine — a reference implementation, not a finished game. Every system above is small enough to read start-to-finish in a few minutes; that's on purpose.